

ЛАБОРАТОРНАЯ РАБОТА 7

1. Назначение и краткая характеристика работы

В процессе выполнения настоящей работы предстоит ознакомиться с отправкой и приемом сообщений по протоколу MQTT.

Целью работы является приобретение навыков в разработке приложения для передачи сообщений по шаблону «Издатель-подписчик» с использованием средств Python.

2. Задание на подготовку к лабораторной работе

1. Разработать программу, генерирующую последовательность чисел формирующую форму сигнала в соответствии с вариантом и сохраняющую эту последовательность в файл формата csv.
2. Разработать программу, считывающую файл формата csv и отправляющую считанные значения в топик MQTT для публикации сообщения. Каждое сообщение содержит одно значение из csv файла.
3. Разработать программу, которая подписывается на топик MQTT и получает значения из этого топика. Полученные значения отображаются на графике, повторяя форму сигнала из файла csv.

3. Методические указания на подготовке к лабораторной работе

MQTT — это протокол обмена сообщениями IoT по шаблону издатель-подписчик. MQTT находится на уровне прикладного протокола (7-й уровень) модели OSI.

Особенности MQTT:

- асинхронный протокол;
- компактные сообщения;
- работа в условиях нестабильной связи на линии передачи данных;
- поддержка нескольких уровней качества обслуживания (QoS);
- легкая интеграция новых устройств.

Сообщения в MQTT передают между тремя участниками — издателями, брокером и подписчиками:

- издатели отправляют сообщения, например это может быть датчики, установленные в термометрах и других IoT-устройствах.

- Брокер (MQTT-broker) является центральным узлом MQTT, который отвечает за взаимодействие между издателями и подписчиками. Брокер получает данные от издателей, обрабатывает их, передает подписчикам и контролирует доставку. Обычно в качестве брокера выступает серверное ПО (MQTT Server).
- подписчики получают данные от датчиков, например аналитическая система, развернутая в облаке.

Работа с протоколом MQTT может осуществляться с использованием библиотеки `paho`.

Ниже приведены примеры реализации издателя и подписчика. В качестве брокера используется Eclipse Mosquitto.

Пример реализации издателя:

```
import random
import time
from paho.mqtt import client as mqtt_client

broker = 'broker.emqx.io'
port = 1883 # стандартный порт Mosquitto
topic = "iot_data" # чат куда издатель будет присылать сообщения
client_id = f'iot_data-{random.randint(0, 1000)}'

def connect_mqtt():
    def on_connect(client, userdata, flags, rc):
        if rc == 0:
            print("Connected to MQTT Broker!")
        else:
            print("Failed to connect, return code %d\n", rc)

    client = mqtt_client.Client(client_id)
    client.on_connect = on_connect
    client.connect(broker, port)
    return client

def publish(client):
    msg_count = 0
    while msg_count < 10:
        time.sleep(1)
        msg = f"messages: {msg_count}"
        result = client.publish(topic, msg) # публикация указанного
        # сообщения в чате
        status = result[0]
        if status == 0:
            print(f"Send `{msg}` to topic `{topic}`")
        else:
            print(f"Failed to send message to topic {topic}")
        msg_count += 1

def run():
    client = connect_mqtt()
    client.loop_start()
    publish(client)
```

```
if __name__ == '__main__':
    run()
```

Ниже приведен пример реализации подписчика:

```
import random

from paho.mqtt import client as mqtt_client

broker = 'broker.emqx.io'
port = 1883
topic = "iot_data"
client_id = f'iot_data-{random.randint(0, 100)}'

def connect_mqtt() -> mqtt_client:
    def on_connect(client, userdata, flags, rc):
        if rc == 0:
            print("Connected to MQTT Broker!")
        else:
            print("Failed to connect, return code %d\n", rc)

    client = mqtt_client.Client(client_id)
    client.on_connect = on_connect
    client.connect(broker, port)
    return client

def subscribe(client: mqtt_client):
    def on_message(client, userdata, msg):
        print(f"Received `{msg.payload.decode()}` from `{msg.topic}`\n"
              "topic")

    client.subscribe(topic)
    client.on_message = on_message

def run():
    client = connect_mqtt()
    subscribe(client)
    client.loop_forever()

if __name__ == '__main__':
    run()
```

Запустив программы издателя и подписчика, в консолях будут выводиться счетчики высланных и полученных сообщений. Также для проверки отправки пакетов следует открыть один из сетевых адаптеров в Wireshark (например Adapter for loopback traffic capture) и начать с помощью него захватывать отсылаемые по MQTT пакеты.

4. Исходные данные

Таблица 8

Варианты формы сигнала		
№	Форма сигнала	Формула сигнала
1	Синусоида	$y(t) = \sin (wt + \varphi)$
2	Меандр	$y(t) = \text{sign}(\sin (wt + \varphi))$
3	Пилообразный сигнал	$y(t) = \frac{2}{\pi} \sum_{n=1}^{\infty} \frac{(-1)^{n+1}}{n} \sin(nx), x - \pi \notin 2\pi k$
4	Сигнал апериодического звена	$y(t) = k(1 - e^{-\frac{t}{T}})$
5	Сигнал реального дифференцирующего звена	$y(t) = \frac{k}{T} e^{-\frac{t}{T}}$

5. Задание на работу в лаборатории

1. Запуск программы для считывания csv файла и публикация сообщений.
2. Запуск программы для подписки на сообщения и отображения графика.

6. Задание к составлению исполнительного отчета

Исполнительный отчет должен включать в себя:

1. титульный лист;
2. выводимые в топике графики;
3. листинг кода;
4. выводы о проделанной работе.